



Resumen clase 16/10 - Sistema de Archivos II

File System

Definición

Responsabilidades

Tipos de File System

Preguntas clave

Conceptos básicos

Bloques y sectores

Definición

Clusters

Definición

¿Por qué existen?

Ejemplo

Ventajas y desventajas

Archivos

Definición

Operaciones básicas

Técnicas de asignación

1. Asignación contigua

Idea

Ejemplo

Ventajas

Desventajas

Uso

2. Asignación encadenada (linked allocation)

Idea

Ejemplo

Ventajas

Desventajas

Variante: **FAT (File Allocation Table)**

3. Asignación indexada

Idea

Ejemplo

Ventajas

Desventajas

<u>Implementación en Unix</u>	
<u>Comparación general</u>	
<u>Representación interna de archivos</u>	
<u>Formato FAT (File Allocation Table)</u>	
<u>Definición</u>	
<u>Estructura general</u>	
<u>Componentes principales</u>	
<u>Tipos de FAT</u>	
<u>Ventajas</u>	
<u>Desventajas</u>	
<u>Conclusión</u>	
<u>Inodos</u>	
<u>Definición</u>	
<u>Contenido de un inodo</u>	
<u>Punteros de un inodo</u>	
<u>Tipos de EXT</u>	
<u>Ventajas</u>	
<u>Desventajas</u>	
<u>Diferencia entre inodo y entrada de directorio</u>	
<u>Ejemplo práctico</u>	
<u>Inodos y Links</u>	
<u>Relación con otros componentes</u>	
<u>Comparación entre FAT con sistemas modernos (Ext2/Ext3)</u>	
<u>Estructuras del File System</u>	
<u>Organización jerárquica</u>	
<u>Estructura en árbol (directorios y subdirectorios)</u>	
<u>Ejemplo de jerarquía</u>	
<u>Propiedades</u>	
<u>Representación interna</u>	
<u>Uso de links</u>	
<u>Definición</u>	
<u>Hard links</u>	
<u>Symbolic links (symlinks)</u>	
<u>Punto de montaje</u>	
<u>Definición</u>	
<u>Ejemplo</u>	
<u>Propiedades</u>	
<u>Comandos útiles</u>	

File System

Definición

- El **sistema de archivos** es un módulo dentro del kernel que permite **administrar y organizar los archivos** dentro de un medio de almacenamiento.
 - El **SO no interpreta el contenido** de un archivo; solo garantiza su **almacenamiento y posterior recuperación**.
 - Dependiendo del sistema operativo, puede soportar **uno o más tipos de File System**.
-

Responsabilidades

- Determinar **cómo se organizan los archivos**:
 - **Internamente**: cómo se estructura la información dentro de un archivo (por ejemplo, Windows y Unix usan secuencias de bytes; la responsabilidad de interpretar el contenido recae en el usuario).
 - **Externamente**: cómo se ordenan los archivos dentro del sistema. Hoy en día todos los FS usan **directorios** → organización **jerárquica en forma de árbol**.
 - Establecer **cómo se nombran los archivos**:
 - Separadores de directorio (/ o \).
 - Uso de extensiones.
 - Longitud máxima y caracteres permitidos.
 - Sensibilidad a mayúsculas/minúsculas.
 - Prefijo o no del dispositivo donde se encuentran.
 - Uso de **puntos de montaje**.
-

Tipos de File System

- **Locales**: FAT, ext2/ext3/ext4, NTFS, XFS, ZFS.
 - **De red / distribuidos**: NFS, AFS, Coda.
 - **Virtuales**: VFS (capa abstracta que permite usar múltiples FS).
 - **Con / sin journaling**: según cómo manejan la consistencia ante fallos.
-

Preguntas clave

- ¿Qué estructuras de datos lo organizan?

- ¿Qué ocurre al abrir, leer o escribir un archivo?
 - ¿Qué se modifica en disco en cada operación?
-

Conceptos básicos

Bloques y sectores

Definición

- Las lecturas y escrituras a un medio de almacenamiento se hacen en **unidades lógicas llamadas bloques**.
 - Cada bloque comprende varios **sectores** (por ejemplo, de 512 o 4096 bytes).
 - Los bloques están numerados secuencialmente desde 0 → **LBA (Logical Block Address)**.
-

Clusters

Definición

Un **cluster** (también llamado “unidad de asignación”) es un **grupo de bloques contiguos** del disco que el sistema de archivos trata como **una sola unidad lógica de almacenamiento**.

En lugar de leer o escribir bloque por bloque, el sistema opera sobre varios bloques juntos.

¿Por qué existen?

Las operaciones de E/S son costosas, especialmente cuando hay muchas operaciones pequeñas.

Para reducir esa sobrecarga:

- El FS agrupa varios bloques físicos consecutivos en un **cluster**.
- Cada acceso lee/escribe más datos de una vez.
- Se **disminuye la fragmentación y mejora la eficiencia del acceso secuencial**.

Ejemplo

- Tamaño de bloque físico: 512 bytes

- El FS define clusters de 8 bloques $\rightarrow 8 \times 512 = \mathbf{4096 \text{ bytes (4 KB)}}$

Cada acceso al disco leerá/escribirá **4 KB por vez**, no 512 bytes.

Ventajas y desventajas

Ventajas

- Menos operaciones de E/S $\rightarrow$ mejor rendimiento.
- Archivos pequeños mejoran su acceso en caché.
- Accesos secuenciales más eficientes.

Desventajas

- **Slack space**: si un archivo ocupa menos que el tamaño del cluster, se desperdicia espacio.

Ejemplo: archivo de 1 KB en un cluster de 4 KB $\rightarrow$ se pierden 3 KB.

Uso en la práctica

- **FAT** usa clusters como unidad mínima de asignación.
- En **Unix (ext2/ext3/ext4)**, el término equivalente es *bloque de datos* (aunque puede haber agrupaciones similares).

Archivos

Definición

- Un **archivo** es una **secuencia de bytes sin estructura**.
- Se lo identifica mediante un **nombre** (que puede incluir una extensión).
- Tiene **metadatos** asociados: nombre, tamaño, permisos, timestamps, etc.

Operaciones básicas

- Crear
- Abrir (`open()`)
- Leer (`read()`)
- Escribir (`write()`)
- Borrar (`unlink()`)

Técnicas de asignación

El sistema de archivos debe decidir **cómo asignar los bloques o clusters** a cada archivo, buscando eficiencia y flexibilidad.

1. Asignación contigua

Idea

- Cada archivo ocupa un **conjunto de bloques consecutivos** en el disco.
- La dirección del primer bloque y la cantidad total permiten acceder a todo el archivo.

Ejemplo

Archivo A → bloques 40–46
Archivo B → bloques 47–50

Ventajas

- Acceso **muy rápido** (secuencial y aleatorio).
- Implementación simple.

Desventajas

- **Fragmentación externa.**
- Difícil **crecimiento dinámico** del archivo.
- Requiere conocer el tamaño al crear el archivo.

Uso

- Ideal para medios **de solo lectura** (CD-ROM, ISO).
-

2. Asignación encadenada (linked allocation)

Idea

- Cada bloque contiene un **puntero al siguiente bloque** del archivo.
- Los bloques no necesitan estar contiguos físicamente.

Ejemplo

Archivo A → [40] → [62] → [15] → [33] → EOF

Ventajas

- No hay fragmentación externa.
- Fácil de expandir.

Desventajas

- Acceso **solo secuencial**.
- Si se pierde un puntero → se pierde el resto del archivo.
- Los punteros ocupan espacio dentro de los bloques.

Variante: **FAT (File Allocation Table)**

- Los punteros están en una tabla global (no en los bloques).

FAT:
[40] → 62
[62] → 15
[15] → 33
[33] → EOF

- **Ventajas:** acceso más rápido.
- **Desventajas:** tabla grande → poco escalable.

3. Asignación indexada

Idea

- Cada archivo tiene una **tabla de punteros (índice)** con las direcciones de todos sus bloques.
- Esta tabla se llama **inodo** (en Unix).

Ejemplo

Inodo del archivo A:

[40, 62, 15, 33]

Ventajas

- Acceso directo a cualquier bloque.
- Sin fragmentación externa.
- Tabla separada → ordenado y flexible.

Desventajas

- Requiere **espacio adicional** para el índice.
- Para archivos grandes, se necesitan índices jerárquicos (directos, indirectos, etc.).

Implementación en Unix

- Cada inodo contiene:
 - **12 punteros directos**
 - **1 indirecto simple**
 - **1 indirecto doble**
 - **1 indirecto triple**

Comparación general

Característica	Contigua	Encadenada (FAT)	Indexada (Inodo)
Acceso secuencial	Excelente	Bueno	Muy bueno
Acceso aleatorio	Excelente	Malo	Bueno
Fragmentación externa	Sí	No	No
Crecimiento del archivo	Difícil	Fácil	Fácil
Eficiencia en disco	Alta	Media	Alta
Robustez	Baja	Media	Alta

Representación interna de archivos

Formato FAT (File Allocation Table)

Definición

- **FAT (File Allocation Table)** es un **sistema de archivos** desarrollado originalmente por **Microsoft** para MS-DOS y luego usado en Windows (FAT12, FAT16, FAT32).
- Es una implementación de la **asignación encadenada**, donde los punteros a los bloques se almacenan en una **tabla central** y no dentro de los bloques mismos.
- Fue uno de los sistemas de archivos más difundidos por su **simplicidad** y **compatibilidad universal** (aún se usa en pendrives, cámaras, SD cards, etc.).

Estructura general

El disco se divide en cuatro regiones principales:



Estructura de un sistema de archivos FAT32.

Componentes principales

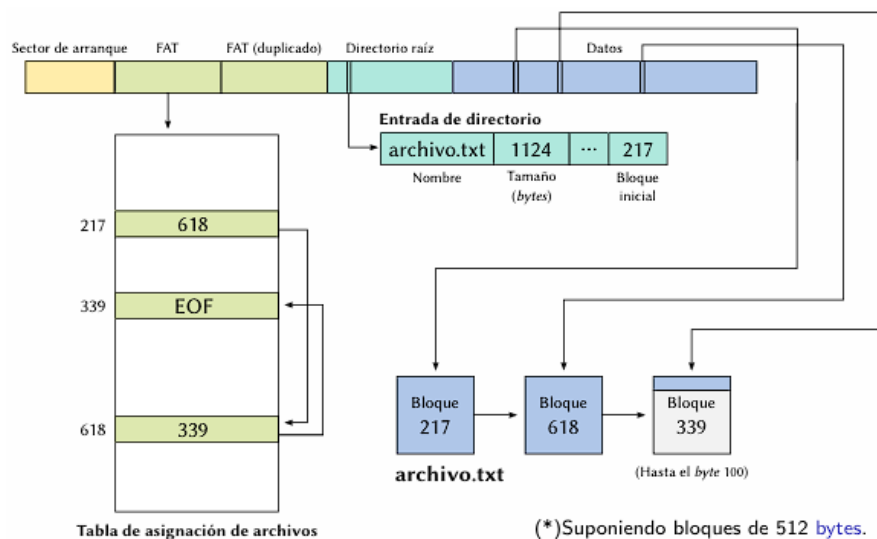
1. Boot Sector

- Contiene información sobre el FS:
 - Tamaño de sector y clúster.
 - Cantidad de copias de la FAT.
 - Número de sectores reservados.
 - Tipo de FAT (12, 16 o 32 bits).
- Es leído por el sistema al montar el FS.

2. FAT (File Allocation Table)

- Tabla que actúa como **índice global** de todos los bloques o clusters del disco.
- Cada entrada de la FAT corresponde a **un cluster**.
- Cada entrada contiene:
 - El **número del siguiente cluster** del archivo, o
 - Un valor especial:
 - **0** → cluster libre.
 - **0xFFF7** → cluster dañado.

- **0xFFFF** → fin de archivo (EOF).



3. Root Directory

- Tabla de entradas con:
 - Nombre del archivo (8.3: 8 caracteres + extensión de 3).
 - Atributos (sistema, oculto, lectura, etc.).
 - Fecha y hora de creación/modificación.
 - Bloque inicial del archivo.
 - Tamaño en bytes.
- En FAT12 y FAT16, el directorio raíz tiene **tamaño fijo**.
- En FAT32, puede ocupar **múltiples bloques** como cualquier otro directorio.

Ejemplo de entrada de directorio:

Nombre	Ext	Atributos	Cluster inicial	Tamaño (bytes)
TAREA	TXT	Archivo	4	1024
FOTO	JPG	Archivo	20	8192
MUSICA	MP3	Archivo	100	24576

4. Data Area

- Contiene los **clusters de datos reales** de los archivos.
- El orden de acceso se determina recorriendo la FAT desde el cluster inicial de cada archivo.

Tipos de FAT

Variante	Tamaño de entrada	Límite máx. de clusters	Tamaño máx. de volumen	Comentarios
FAT12	12 bits	~4.000	32 MB	Usado en disquetes.
FAT16	16 bits	~65.000	2 GB	Usado en DOS/Windows 95.
FAT32	28 bits útiles	~268 millones	2 TB	Usado en Windows modernos, pendrives, SD.

Ventajas

- Estructura simple y de fácil implementación.
- Amplia compatibilidad (Windows, Linux, cámaras, BIOS, etc.).
- Buena performance en discos pequeños o medios removibles.
- Fácil recuperación de datos ante errores leves.

Desventajas

- La FAT debe mantenerse completa en memoria → **no escala bien** a discos grandes.
- **Fragmentación externa e interna** frecuentes.
- Sin control de permisos ni usuarios.
- No soporta journaling (riesgo ante cortes de energía).
- Tamaño máximo de archivo:
 - FAT16 → 2 GB
 - FAT32 → 4 GB

Conclusión

- **FAT** fue clave históricamente por su **simplicidad y compatibilidad universal**, pero **carece de las características modernas** necesarias en sistemas multitarea o multiusuario.
- Por eso, fue reemplazado progresivamente por **NTFS** (Windows) y **Ext2/Ext3/Ext4** (Unix/Linux), que usan **inodos** y **asignación indexada**.

Inodos

Definición

- Un **inodo (index node)** es una **estructura de datos** utilizada por los sistemas de archivos tipo **Unix** para representar a cada archivo o directorio.
- Contiene **toda la información sobre el archivo**, excepto su nombre.
- Cada archivo tiene **un inodo único**, identificado por un **número de inodo (inode number)**.

El nombre del archivo se almacena en el directorio, pero el directorio solo guarda el nombre y el número de inodo asociado.

Contenido de un inodo

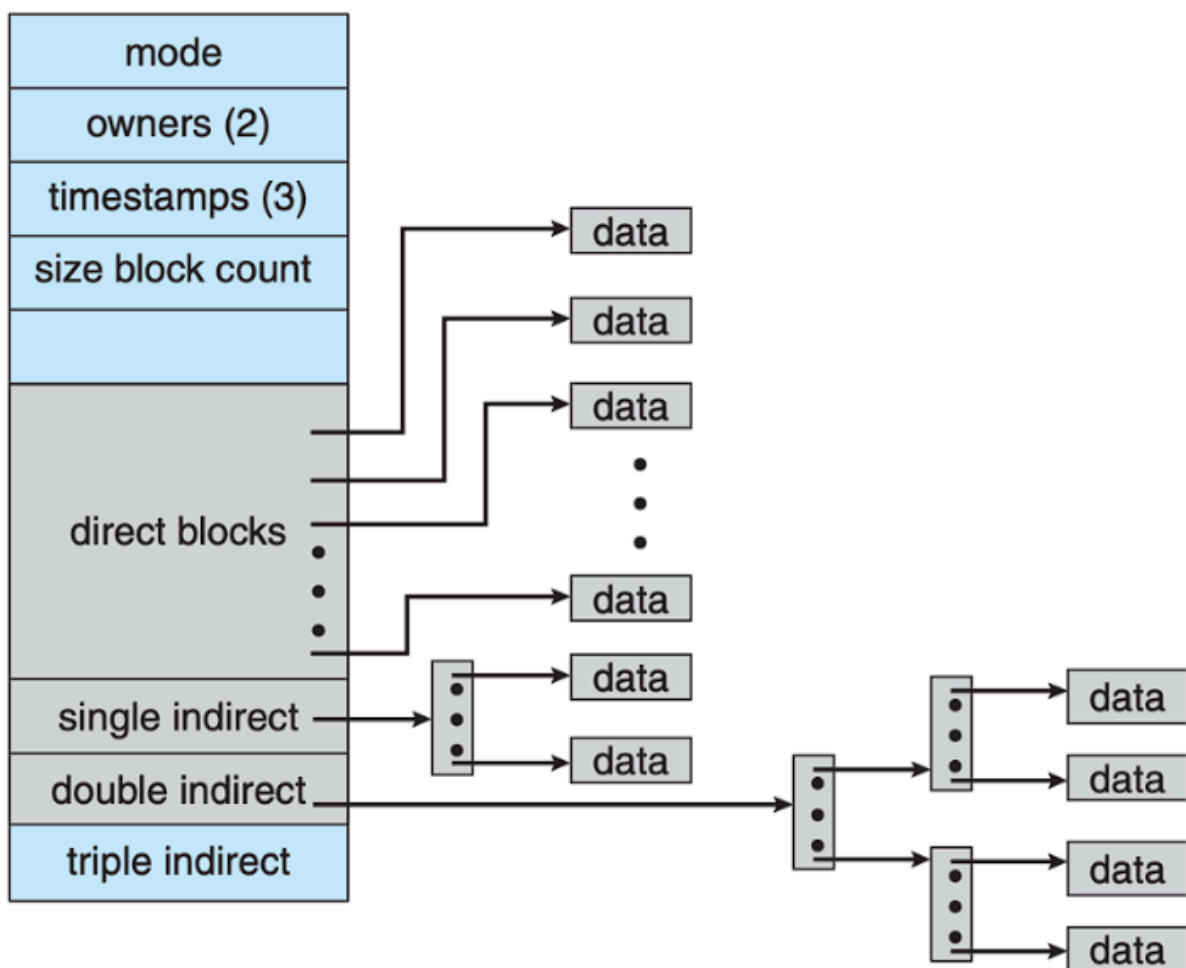
Cada inodo guarda:

- **Metadatos del archivo:**
 - Tipo de archivo (regular, directorio, dispositivo, enlace, etc.).
 - Permisos (rwx, bits especiales).
 - UID (usuario propietario) y GID (grupo).
 - Tamaño del archivo.
 - Timestamps:
 - *atime* → último acceso.
 - *mtime* → última modificación de contenido.
 - *ctime* → último cambio en metadatos.
 - Cantidad de enlaces (*link count*).
- **Punteros a los bloques de datos:**
 - Indican dónde están almacenados los datos reales en disco.

Punteros de un inodo

Para permitir archivos de distinto tamaño, un inodo tiene varios niveles de punteros:

Tipo de puntero	Qué apunta	Alcance
Directos	Bloques de datos del archivo	Rápido acceso a archivos pequeños
Indirecto simple	Bloque que contiene punteros a otros bloques de datos	Archivos medianos
Indirecto doble	Bloque que apunta a bloques de punteros (cada uno con referencias a datos)	Archivos grandes
Indirecto triple	Bloque que apunta a bloques de dobles indirectos	Archivos muy grandes



Esto permite manejar archivos de cualquier tamaño sin que el inodo tenga que almacenar todos los punteros directamente.

Tipos de EXT

Característica	Ext1	Ext2	Ext3	Ext4
Año de aparición	1992	1993	2001	2008

Característica	Ext1	Ext2	Ext3	Ext4
Journaling	✗ No	✗ No	✓ Sí	✓ Sí (mejorado con checksums)
Extents (bloques contiguos)	✗ No	✗ No	✗ No	✓ Sí
Compatibilidad hacia atrás	—	—	✓ con Ext2	✓ con Ext2/3
Tamaño máx. de archivo	64 MB	2 TB	2 TB	16 TB
Tamaño máx. de FS	2 GB	32 TB	32 TB	1 EB
Fragmentación	Alta	Media	Media	Baja
Recuperación tras fallos	Muy lenta	Lenta (fsck)	Rápida (journal)	Muy rápida (journal + extents)
Soporte de timestamps extendidos	✗	✗	✗	✓ (nanosegundos, año 2514)
Asignación de bloques	Contigua	Indexada	Indexada con journaling	Extents + delayed allocation
Uso actual	Obsoleto	Obsoleto	Aún en uso en sistemas antiguos	Predominante en Linux modernos

Ventajas

- Permite acceder rápidamente a archivos pequeños (directos).
- Escala fácilmente a archivos grandes (indirectos).
- Cada archivo tiene su propio inodo → sin tabla global como en FAT.
- Menor contención entre procesos concurrentes.
- Fácil detección y recuperación de inconsistencias.

Desventajas

- Al crecer mucho el archivo, las lecturas indirectas añaden saltos extra.
- Si el sistema se corrompe, puede haber **inodos huérfanos** (bloques asignados sin referencia desde ningún directorio).

- Se necesita un espacio reservado para la **tabla de inodos**, que no puede crecer dinámicamente (en ext2/ext3).

Diferencia entre inodo y entrada de directorio

Concepto	Qué contiene	Dónde está
Inodo	Metadatos + punteros a bloques	Tabla de inodos
Entrada de directorio	Nombre del archivo + número de inodo	Dentro del directorio correspondiente

Esto permite que múltiples nombres (hard links) apunten al mismo inodo.

Ejemplo práctico

Supongamos el archivo `/home/ariel/docs/tesis.pdf`:

1. El sistema busca el inodo del directorio raíz `/`.
2. Desde allí encuentra el inodo de `home`.
3. Luego el de `ariel`, después `docs`.
4. Finalmente encuentra la entrada `"tesis.pdf"` → número de inodo: 20483.
5. El inodo 20483 contiene:
 - Tamaño: 350 KB
 - UID: 1000
 - Punteros directos/indirectos a los bloques de datos en disco.

Inodos y Links

- **Hard link** → varios nombres apuntan al mismo inodo.
 - El contador de enlaces (*link count*) del inodo se incrementa.
 - Solo se libera el espacio cuando este contador llega a 0.
- **Symbolic link** → apunta a una ruta, no al inodo directamente.
 - Tiene su propio inodo (con el contenido de la ruta destino).

Relación con otros componentes

- Los **directorios** almacenan los pares (*nombre, número de inodo*).
- Los **bloques de datos** contienen el contenido real del archivo.

- El **superbloque** del FS mantiene información global (cantidad de inodos, tamaño de bloques, etc.).
- Los **group descriptors** (en Ext2/Ext3) agrupan inodos y bloques de datos físicamente.

Comparación entre FAT con sistemas modernos (Ext2/Ext3)

Característica	FAT	Ext2 / Ext3
Estructura de punteros	Tabla global	Inodos individuales
Escalabilidad	Limitada	Alta
Permisos / usuarios	No	Sí (rwx, UID, GID)
Fragmentación	Alta	Baja
Journaling	No	Sí (Ext3/4)
Robustez ante fallos	Baja	Alta

Estructuras del File System

Organización jerárquica

Estructura en árbol (directorios y subdirectorios)

Los sistemas de archivos modernos organizan la información en una **estructura jerárquica** con forma de **árbol**, donde:

- Cada **nodo** representa un archivo o un directorio.
- El **directorio raíz** (/) es el punto de partida del árbol.
- Cada **directorio** puede contener archivos y otros subdirectorios.

Ejemplo de jerarquía

```

/
├── home/
│   ├── ariel/
│   │   ├── documentos/
│   │   │   └── tesis.pdf
│   │   └── fotos/

```

```
| | | — viaje.jpg
| | — milagros/
| — etc/
| — config.txt
```

Propiedades

- Cada archivo tiene una **ruta absoluta única** (e.g. `/home/ariel/documentos/tesis.pdf`).
- Los **directorios son archivos especiales** que contienen pares (*nombre, referencia a inodo*).
- En Unix, los directorios suelen implementarse como una **lista o tabla de entradas**.

Representación interna

- Cada archivo o directorio tiene su propio **inodo** (estructura que almacena metadatos y punteros a los bloques).
- Los directorios contienen una **lista de nombres** y los **números de inodo** asociados.
- El sistema traduce rutas como `/usr/bin/python3` recorriendo los directorios paso a paso, resolviendo los inodos intermedios hasta llegar al archivo final.

Uso de links

Definición

- Un **link** es un alias, otro nombre que referencia el mismo archivo.
- Al introducir links, la estructura deja de ser estrictamente un árbol y pasa a ser un **grafo dirigido** (pueden existir ciclos).

Hard links

- Dos nombres distintos apuntan al **mismo inodo**.
- Si se borra uno de los nombres, el archivo permanece mientras existan otros enlaces.

Symbolic links (symlinks)

- Son **archivos especiales** que contienen una **ruta** hacia otro archivo o directorio.

- Pueden apuntar a ubicaciones inexistentes o en otros sistemas de archivos.
 - Funcionan como un "atajo".
-

Punto de montaje

Definición

Un **punto de montaje** es el **lugar dentro del árbol de directorios** donde se integra otro **sistema de archivos** (por ejemplo, otra partición o disco).

Ejemplo

```
sudo mount /dev/sdb1 /home/ariel/usb
```

El contenido del disco `/dev/sdb1` se hace accesible en la carpeta `/home/ariel/usb`.

Propiedades

- Permite **unificar múltiples sistemas de archivos** bajo un solo árbol (`/`).
- No usa letras de unidad (como C:, D: en Windows).
- Antes de montar, la carpeta puede estar vacía; luego muestra el contenido del nuevo FS.

Comandos útiles

- `mount` → lista los sistemas de archivos montados.
- `umount` → desmonta un FS.
- `/etc/fstab` → define montajes automáticos al iniciar el sistema.